

NETWORK TRAFFIC ANALYSIS

Investigation Report — Wireshark Lab Challenge

Analyst	Paul
Track	SOC Track: AltSchool Africa
Platform	Kali Linux (tshark CLI)
Capture	wireshark_challenge.pcap: SOC 101 Course Challenges
Endpoint	172.16.1.16 (BulbaTech Innovations SOC Alert)
Tools Used	tshark · xxd · sha256sum · capinfos · VirusTotal

Scenario

The SOC at BulbaTech Innovations received an alert of abnormal traffic patterns and a high number of repeated queries originating from one of their external-facing endpoints (172.16.1.16). A packet capture was obtained for analysis. This report documents each challenge question, the exact tshark/CLI command used to investigate it on Kali Linux, the reasoning behind the command, the raw terminal output, and the derived answer.

Q1 How many total packets are in the capture?

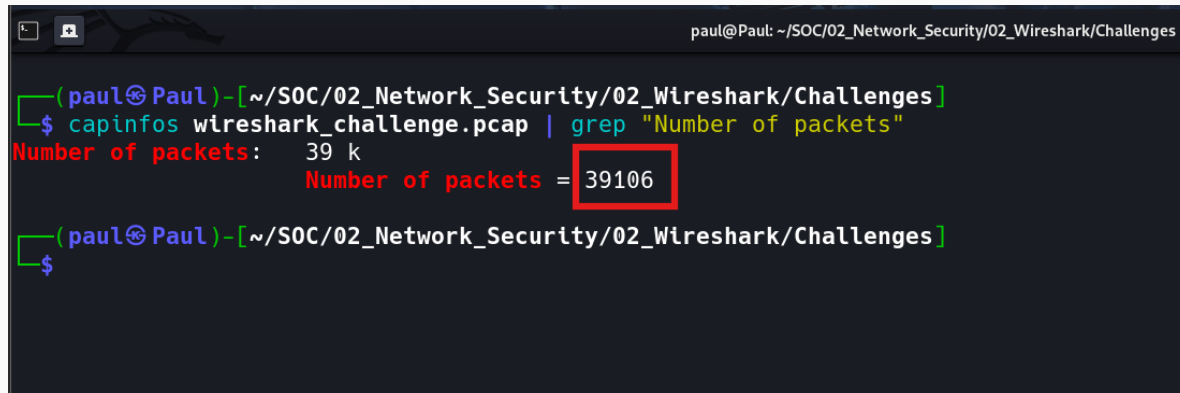
COMMAND

```
capinfos wireshark_challenge.pcap | grep "Number of packets"
```

HOW IT WORKS

capinfos is a Wireshark utility that reads the PCAP global header and reports file-level statistics without parsing every packet. It is significantly faster than tshark for a simple count. The grep filters its output down to the single relevant line. This is the correct tool for this question — using tshark | wc -l would count displayed lines, which can be inaccurate if packets are truncated or malformed.

SCREENSHOT



```
paul@Paul: ~/SOC/02_Network_Security/02_Wireshark/Challenges
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ capinfos wireshark_challenge.pcap | grep "Number of packets"
Number of packets: 39 k
Number of packets = 39106
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

Q2 What was the first domain name queried and resolved? | What is its IP?

COMMAND

```
tshark -r wireshark_challenge.pcap \
  -Y "dns.flags.response == 1" \
  -T fields \
  -e frame.number \
  -e dns.qry.name \
  -e dns.a \
  | head -20
```

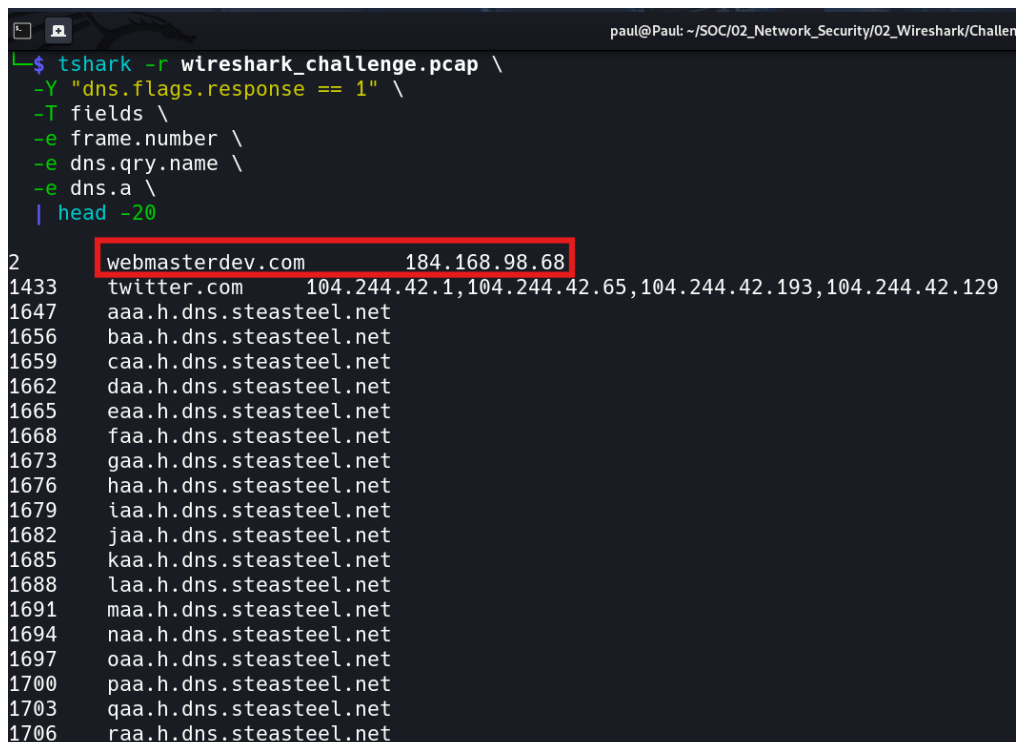
HOW IT WORKS

The filter `dns.flags.response == 1` isolates DNS response packets only — these are server replies where a query was successfully answered. If we filtered on `dns.flags.response == 0` we would see queries but not know whether they resolved. Extracting `dns.qry.name` gives the domain that was asked for, and `dns.a` gives the returned IPv4 A record. Piping to `head -20` shows the first 20 results. The very first row corresponds to the earliest resolved domain in the entire capture.

💡 A 'resolved' query means you received a valid A record back — not an NXDOMAIN (non-existent domain) error. That distinction matters here.

READING THE OUTPUT

Frame 2 is the earliest response — `webmasterdev.com` resolved to `184.168.98.68`. Twitter at frame 1433 is normal background traffic. The sequential `aaa/baa/caa` pattern starting at frame 1647 with no IP returned is immediately suspicious — this is automated beacon traffic, not a human browsing the web.



```
paul@Paul: ~/SOC/02_Network_Security/02_Wireshark/Challen
$ tshark -r wireshark_challenge.pcap \
  -Y "dns.flags.response == 1" \
  -T fields \
  -e frame.number \
  -e dns.qry.name \
  -e dns.a \
  | head -20
2      webmasterdev.com      184.168.98.68
1433   twitter.com          104.244.42.1,104.244.42.65,104.244.42.193,104.244.42.129
1647   aaa.h.dns.steasteel.net
1656   baa.h.dns.steasteel.net
1659   caa.h.dns.steasteel.net
1662   daa.h.dns.steasteel.net
1665   eaa.h.dns.steasteel.net
1668   faa.h.dns.steasteel.net
1673   gaa.h.dns.steasteel.net
1676   haa.h.dns.steasteel.net
1679   iaa.h.dns.steasteel.net
1682   jaa.h.dns.steasteel.net
1685   kaa.h.dns.steasteel.net
1688   laa.h.dns.steasteel.net
1691   maa.h.dns.steasteel.net
1694   naa.h.dns.steasteel.net
1697   oaa.h.dns.steasteel.net
1700   paa.h.dns.steasteel.net
1703   qaa.h.dns.steasteel.net
1706   raa.h.dns.steasteel.net
```

Q3 How many HTTP packets are contained in the capture?

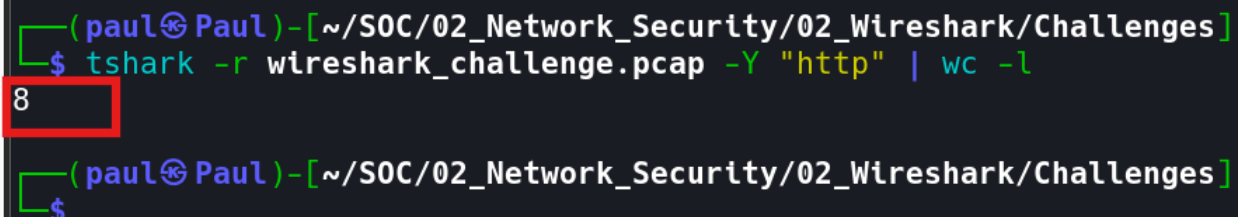
COMMAND

```
tshark -r wireshark_challenge.pcap -Y "http" | wc -l
```

HOW IT WORKS

The display filter http matches any packet that tshark identifies as containing an HTTP layer — this includes GET requests, POST requests, server responses (200 OK, 404 etc.), and continuation data frames. Piping to wc -l counts the number of output lines, each of which represents one matched packet. This is more precise than filtering tcp.port == 80, which would match raw TCP traffic on port 80 even before the HTTP handshake completes.

SCREENSHOT



```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ tshark -r wireshark_challenge.pcap -Y "http" | wc -l
8
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

Q4 What is the relative path the victim accessed to download a file?

COMMAND

```
tshark -r wireshark_challenge.pcap \
  -Y "http.request" \
  -T fields \
  -e http.request.method \
  -e http.request.uri \
  -e http.host
```

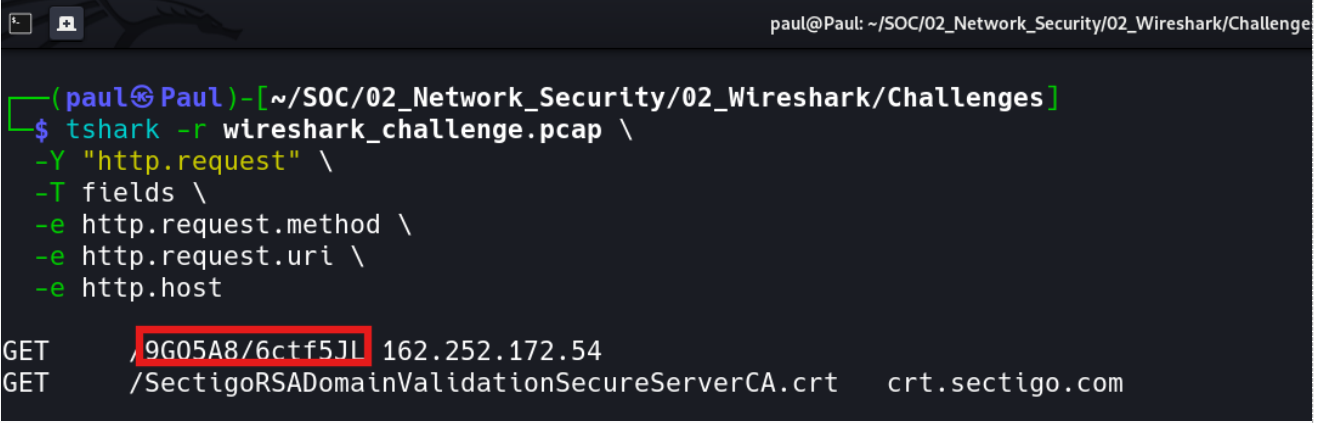
HOW IT WORKS

The filter http.request isolates outbound HTTP request packets. The field http.request.uri extracts the relative path — everything after the hostname in the URL. The field http.host shows who the request was sent to. Together these tell us exactly what the victim's machine requested and from where. The 'relative path' in the question means just the URI portion — not the full URL.

READING THE OUTPUT

Line 1 is highly suspicious: the path /9GQ5A8/6ctf5JL has no file extension (obfuscated), and it was served from a bare IP address (162.252.172.54) rather than a domain — deliberately bypassing URL reputation checks and domain-based detection. Line 2 is a legitimate SSL certificate fetch from Sectigo, a known certificate authority — normal TLS background traffic, can be ignored.

SCREENSHOT



```
paul@Paul: ~/SOC/02_Network_Security/02_Wireshark/Challenge

(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ tshark -r wireshark_challenge.pcap \
  -Y "http.request" \
  -T fields \
  -e http.request.method \
  -e http.request.uri \
  -e http.host

GET      /9G05A8/6ctf5JL 162.252.172.54
GET      /SectigoRSADomainValidationSecureServerCA.crt crt.sectigo.com
```

Q5 Based on the response header, what file type does the server claim the downloaded file to be?

COMMAND

```
tshark -r wireshark_challenge.pcap \
  -Y "http.response" \
  -T fields \
  -e http.content_type
```

HOW IT WORKS

When a web server responds to a GET request, it includes a Content-Type header that declares the MIME type of the file being sent. This is what the server tells the client (and any security tool inspecting the traffic) the file format is. Attackers frequently manipulate this header to disguise malware. The filter `http.response` isolates server reply packets, and `http.content_type` extracts that specific header value.

💡 The Content-Type header is a declaration, not a guarantee. Any server can claim any type — which is why we verify with magic bytes in Q7.

SCREENSHOT

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]  
$ tshark -r wireshark_challenge.pcap \  
-Y "http.response" \  
-T fields \  
-e http.content_type  
image/gif  
application/pkix-cert
```

Q6 What is the actual file signature (magic bytes) of the downloaded file?

STEP 1 — EXPORT HTTP OBJECTS FROM THE PCAP

```
tshark -r wireshark_challenge.pcap --export-objects http,./exported_files
```

STEP 2 — LIST EXTRACTED FILES

```
ls -lh exported_files/
```

STEP 3 — INSPECT THE PAYLOAD

```
file exported_files/6ctf5JL  
xxd exported_files/6ctf5JL | head -3
```

HOW IT WORKS

The `--export-objects` flag instructs tshark to reconstruct files that were transferred over HTTP by reassembling TCP stream data, then save each object to disk.

After listing the exported directory, one file stands out **6ctf5JL** at 1.3MB:- far larger than all others (~500 bytes).

Running `xxd` dumps the file's raw binary content as hexadecimal.

The very first bytes known as 'magic bytes' or a 'file signature' identify the actual file format regardless of its name, extension, or Content-Type header.

💡 Every file format reserves its first bytes as a signature. The OS and security tools use these bytes — not the filename — to determine what a file actually is. You cannot trust the extension or the server's Content-Type header.

TERMINAL OUTPUT — XXD

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ file exported_files/6ctf5JL
xxd exported_files/6ctf5JL | head -3

exported_files/6ctf5JL: PE32 executable for MS Windows 6.00 (DLL), Intel i386, 6 sections
00000000: 4d5a 9000 0300 0000 0400 0000 ffff 0000  MZ .....
00000010: b800 0000 0000 0000 4000 0000 0000 0000  .....@.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000  .....

(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

READING THE OUTPUT

The first two bytes are 0x4D 0x5A — which in ASCII is the text 'MZ'.

This is the Windows PE (Portable Executable) magic number, present at byte 0 of every Windows .exe and .dll file. The right-hand column of xxd shows the ASCII representation.

The server lied about this file's type. In reality it is a Windows executable delivered as malware.

Hex Bytes	ASCII	File Type
4D 5A	MZ	Windows PE Executable (.exe / .dll) ← THIS FILE
7F 45 4C 46	.ELF	Linux ELF Executable
25 50 44 46	%PDF	PDF Document
50 4B 03 04	PK..	ZIP / Office File (.zip / .docx / .xlsx)
FF D8 FF	—	JPEG Image
89 50 4E 47	.PNG	PNG Image

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ ls -lh exported_files/
total 4.6M
-rw-r--r-- 1 paul paul 1.3M May  9 19:04 6ctf5JL
-rw-r--r-- 1 paul paul 553 May  9 19:04 object24044
-rw-r--r-- 1 paul paul 562 May  9 19:04 object24233
-rw-r--r-- 1 paul paul 544 May  9 19:04 object24402
-rw-r--r-- 1 paul paul 710 May  9 19:04 object24427
-rw-r--r-- 1 paul paul 544 May  9 19:04 object24580
-rw-r--r-- 1 paul paul 515 May  9 19:04 object24593
-rw-r--r-- 1 paul paul 515 May  9 19:04 object24781
-rw-r--r-- 1 paul paul 544 May  9 19:04 object24938
-rw-r--r-- 1 paul paul 515 May  9 19:04 object24984
-rw-r--r-- 1 paul paul 1.5K May  9 19:04 object24992
-rw-r--r-- 1 paul paul 457 May  9 19:04 object24995
```

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ file exported_files/6ctf5JL
xxd exported_files/6ctf5JL | head -3

exported_files/6ctf5JL: PE32 executable for MS Windows 6.00 (DLL), Intel i386, 6 sections
00000000: 4d5a 9000 0300 0000 0400 0000 ffff 0000 MZ!.....
00000010: b800 0000 0000 0000 4000 0000 0000 0000 .....@.....
00000020: 0000 0000 0000 0000 0000 0000 0000 0000 .....

(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

Q7 What command-line utility was used by the victim to download the file?

COMMAND

```
tshark -r wireshark_challenge.pcap \
-Y "http.request" \
-T fields \
-e http.user_agent
```

HOW IT WORKS

Every HTTP client — whether a browser, curl, wget, or PowerShell — identifies itself in the User-Agent request header. This header is sent with every outbound HTTP request.

Extracting http.user_agent from the GET request that fetched the payload reveals the exact tool the victim's machine used. This is critical forensic evidence: if the endpoint is a standard workstation, seeing a PowerShell User-Agent downloading an executable from a bare IP is a major red flag — it indicates script-based execution, not a user clicking a browser link.

COMMON CLI TOOL SIGNATURES

User-Agent String	Tool
curl/7.x.x	curl
Wget/1.x	wget
PowerShell / WindowsPowerShell	PowerShell (Invoke-WebRequest)
Python-urllib/3.x	Python script
Mozilla/5.0 ...	Browser (Firefox/Chrome)

SCREENSHOT

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ tshark -r wireshark_challenge.pcap \
  -Y "http.request" \
  -T fields \
  -e http.user_agent

Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.22621.963
Microsoft-CryptoAPI/10.0

(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

Q8 What is the SHA256 hash of the downloaded file?

COMMAND

```
sha256sum exported_files/6ctf5JL
```

HOW IT WORKS

SHA256 is a cryptographic hash function that produces a unique 64-character fingerprint of a file. Even a single bit change in the file produces a completely different hash. This makes it the standard identifier for malware samples — it is what you submit to threat intelligence platforms like VirusTotal. Running sha256sum on the extracted file produces this fingerprint from the actual bytes on disk.

💡 Hashing by SHA256 is safer than uploading a file to VirusTotal — you never transmit the binary itself. The hash alone gives you full detection results.

SCREENSHOT

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ sha256sum exported_files/6ctf5JL
9b8ffdc8ba2b2caa485cca56a82b2dcdbd251f65fb30bc88f0ac3da6704e4d3c6 exported_files/6ctf5JL
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$
```

Q9 Based on VirusTotal, what type of malware infected the endpoint?

METHOD

The SHA256 hash was submitted to VirusTotal (<https://www.virustotal.com>) via the Search tab. No file upload was needed — searching by hash retrieves full detection results safely.

HOW IT WORKS

VirusTotal aggregates results from 70+ antivirus engines and assigns a 'Popular threat label' based on the most common detection name across those engines.

The Tags section lists behavioural attributes identified through sandbox execution. The Family labels section identifies which known malware family the sample belongs to.

VIRUSTOTAL RESULTS

Field	Value
Detection Rate	59 / 71 vendors (83%)
Popular Threat Label	trojan.pikabot/mikey
Threat Category	Trojan
Family Labels	pikabot mikey zenpak
Behaviour Tags	pedll spreader overlay checks-user-input detect-debug-environment persistence
File Size	1.23 MB

WHAT IS PIKABOT?

PikaBot is a modular trojan and loader that emerged in early 2023. It is strongly associated with the Black Basta ransomware group as an initial access tool.

It works by loading and executing additional payloads on a compromised machine. Key capabilities reflected in the VirusTotal tags include: anti-debug/sandbox evasion (detect-debug-environment, checks-user-input), persistence across reboots (persistence), lateral movement potential (spreader), and data appended beyond the PE structure to evade signature detection (overlay).

PikaBot uses DNS tunneling as its primary C2 channel directly matching what we observe in the PCAP.

SCREENSHOT

9b8ffdc8ba2b2caa485cca56a82b2dcbd251f65fb30bc88f0ac3da6704e4d3c6

59/71 security vendors flagged this file as malicious

Community Score: 59/71

Size: 1.23 MB | Last Analysis Date: 15 days ago

6ctf5JL

pedll spreader overlay checks-user-input detect-debug-environment persistence

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label: trojan, **pikabot**, mikey | Threat categories: trojan | Family labels: pikabot, mikey, zenpak

Security vendors' analysis

AhnLab-V3	Trojan.Win.Qakbot.R579273	Alibaba	Trojan.Win32/Zenpak.42993aea
AliCloud	Trojan.Win/PikaBot.LH8PHU	ALYac	Trojan.Agent.Pikabot
Antiy-AVL	Trojan/Win32.Zenpak	Arcabit	Trojan.Mikey.D23EB2
Arctic Wolf	Unsafe	Avast	Win32:MalwareX-gen [Trj]
AVG	Win32:MalwareX-gen [Trj]	Avira (no cloud)	TR/Crypt.Agent.Lzdahr
BitDefender	Gen:Variant.Mikey.147122	Bkav Pro	W32.AIDetectMalware
ClamAV	Win.Trojan.BITd.10007206.0	CrowdStrike EDR	Win32:MalwareX-gen.10007206.0

Q10 What protocol makes up the majority of UDP packets?

COMMAND

```
tshark -r wireshark_challenge.pcap \
-Y "udp" \
-T fields \
-e _ws.col.Protocol \
| sort | uniq -c | sort -rn
```

HOW IT WORKS

The filter `udp` isolates all UDP traffic. The special field `_ws.col.Protocol` extracts the highest-level protocol label `tshark` assigns to each packet — for example, DNS, DHCP, SSDP, mDNS. Piping through `sort`, then `uniq -c` (count duplicates), then `sort -rn` (reverse numerical sort) produces a ranked frequency table. The protocol at the top of the list is the dominant UDP protocol in this capture. DNS is the expected answer here given the massive C2 beaconing we observed — but running the command confirms it with actual counts.

SCREENSHOT

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ tshark -r wireshark_challenge.pcap \
  -Y "udp" \
  -T fields \
  -e _ws.col.Protocol \
  | sort | uniq -c | sort -rn

29771 DNS
13 BROWSER
```

Q11 What base domain is continually queried? (defanged format)

COMMAND

```
tshark -r wireshark_challenge.pcap \
  -Y "dns.flags.response == 0" \
  -T fields \
  -e dns.qry.name \
  | sort | uniq -c | sort -rn \
  | head -30
```

HOW IT WORKS

This time we filter `dns.flags.response == 0` to see outbound DNS queries (what the victim is asking for), rather than responses. Ranking by frequency reveals beaconing patterns — any domain queried hundreds or thousands of times is automated behaviour. The base domain is the persistent root across all malicious queries; the changing subdomain labels carry encoded C2 data. Defanging replaces dots with `[.]` so the domain cannot be accidentally clicked or resolved when shared in security reports.

READING THE OUTPUT

The subdomain structure reveals the tunneling mechanism: `rido4` is a session identifier, the hex string `(26fa3eb6)` encodes the actual C2 data, and `dns.steasteel.net` is the constant C2 base domain. The malware queries thousands of different subdomains all resolving back to the same base to exfiltrate data and receive instructions byte by byte inside DNS query labels. No IP is returned because the data flows in the query itself, not the answer.

DEFANGING

To safely share the domain in reports or chat, replace each dot with `[.]`:

```
dns.steasteel.net → dns.steasteel[.]net
```

SCREENSHOT

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ tshark -r wireshark_challenge.pcap \
-Y "dns.flags.response == 0" \
-T fields \
-e dns.qry.name \
| sort | uniq -c | sort -rn \
| head -30
```

```
3699 ridoj4.26fa3eb6.dns.steasteel.net
126 ridoj4.1a3f8ef6.dns.steasteel.net
118 ridoj4.19074dfe.dns.steasteel.net
6 tua.h.dns.steasteel.net
4 xca.h.dns.steasteel.net
4 upb.h.dns.steasteel.net
4 ufb.h.dns.steasteel.net
4 twitter.com
4 tea.h.dns.steasteel.net
4 tba.h.dns.steasteel.net
4 roc.h.dns.steasteel.net
4 qua.h.dns.steasteel.net
```

```
(paul@Paul)-[~/SOC/02_Network_Security/02_Wireshark/Challenges]
$ echo "dns.steasteel.net" | sed 's/\./[.]/g'

dns[.]steasteel[.]net
```

Q12 Read MITRE ATT&CK T1071.004. What attack technique are we seeing?

HOW IT WORKS

MITRE ATT&CK is a globally accessible knowledge base of adversary tactics and techniques. T1071 covers 'Application Layer Protocol' — adversaries communicating using legitimate application-layer protocols to blend in with normal network traffic. Sub-technique .004 specifically covers DNS as the covert channel. This maps directly to what we observed: the malware embedding C2 traffic inside DNS query names, exploiting the fact that port 53 is rarely blocked by firewalls.

MITRE REFERENCE

<https://attack.mitre.org/techniques/T1071/004/>

EVIDENCE MAPPING

T1071.004 Indicator	Observed in PCAP
High query volume	3,699+ queries to a single subdomain pattern
Algorithmic subdomains	aaa/baa/caa sequence then hex-encoded labels
No IP returned	Data is in the query label, not the DNS answer
Single persistent domain	dns.steasteel[.]net constant across all beacons

Unusual query rate

Queries firing every few seconds — automated

✓ **ANSWER: DNS Tunneling** (MITRE ATT&CK T1071.004 — Application Layer Protocol: DNS)

Findings Summary

All findings consolidated from the investigation:

#	Question	Answer
Q1	Total packet count	39106
Q2	First resolved domain	webmasterdev.com
Q3	Associated IP address	184.168.98.68
Q4	Total HTTP packets	[8]
Q5	Relative download path	/9GQ5A8/6ctf5JL
Q6	Server claimed file type	image/gif
Q7	Actual magic bytes	MZ (0x4D 0x5A) — Windows PE
Q8	Download tool / User-Agent	[see http.user_agent output]
Q9	SHA256 hash	9b8ffdc8ba2b...4e4d3c6
Q10	Malware type	Trojan — PikaBot
Q11	Dominant UDP protocol	DNS
Q12	C2 base domain (defanged)	dns[.]steasteel[.]net
Q13	Attack technique	DNS Tunneling — T1071.004

Reconstructed Attack Narrative

Based on the PCAP evidence, the following kill chain was reconstructed:

1	Initial Contact Frame 2: The victim machine resolved webmasterdev.com to 184.168.98.68 — the first attacker-controlled domain in the capture.
2	Payload Delivery An HTTP GET request was issued to /9GQ5A8/6ctf5JL on bare IP 162.252.172.54. No domain name, no file extension — designed to evade URL reputation systems.
3	Disguise Detected The server's Content-Type header misrepresented the file type. xxd inspection revealed the truth: 0x4D 0x5A (MZ) — a Windows PE executable.
4	Malware Confirmed SHA256 hash submitted to VirusTotal: 59/71 vendors flagged the file as trojan.pikabot/mikey — a modular trojan loader associated with the Black Basta ransomware group.

C2 Established

Following execution, PikaBot began DNS tunneling to dns.steasteel[.]net. 3,699+ queries were sent, encoding C2 communications in algorithmically-generated subdomain labels — MITRE ATT&CK T1071.004.